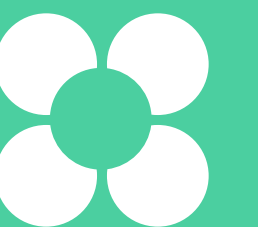


Обработка ошибок в Go

Игорь Шамаев
Руководитель направления разработки



Игорь Шамаев

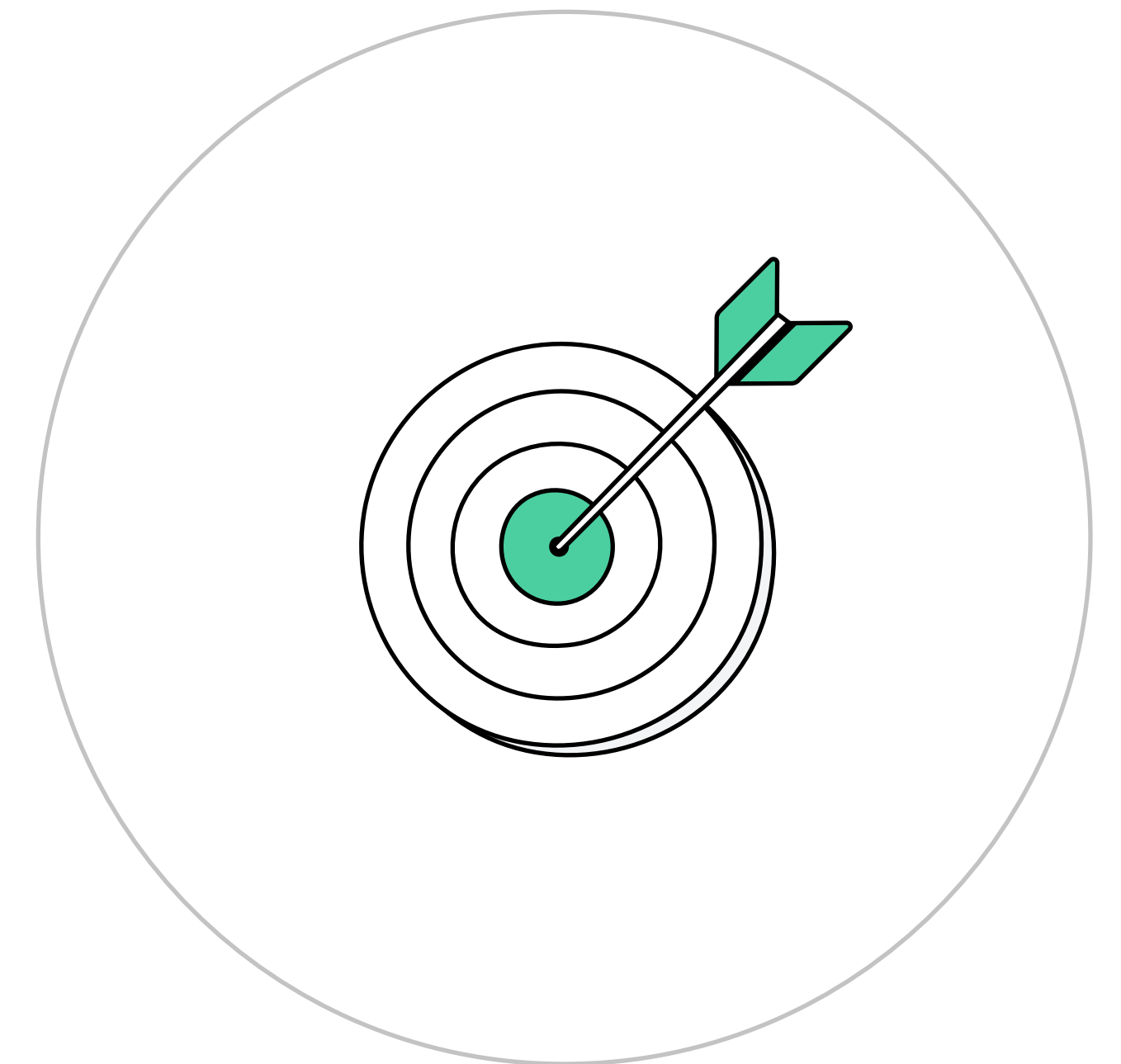
О спикере:

- Руководитель направления разработки в Домклик
- Развивает и поддерживает внутреннюю базу данных управления конфигурацией на Golang и TypeScript
- В прошлом — руководитель команды разработки в Сбере



Цели занятия

- Изучить принципы работы с ошибками в Go
- Понять отличие работы с ошибками в Go от других языков
- Узнать, как правильно обрабатывать ошибки



План занятия

Обработка ошибок в языке Go:
различные способы и их применение в коде



Обработка ошибок в языке Go

Различные способы и их применение в коде

Введение в обработку ошибок

Ошибки в Go — это значения, а не исключения

В отличие от языков вроде Python или Java, где используются исключения, в Go вы сами контролируете ошибки. Это как если бы вы проверяли каждую ступеньку на лестнице перед шагом. Да, это требует больше кода, зато вы не упадёте в пропасть неожиданно

```
data, err := os.ReadFile("file.txt")
if err != nil {
    log.Fatal(err)
}
```


Базовый подход – if err != nil

В Go ошибки возвращаются явно, как обычные значения. Это делает код предсказуемым: вы всегда видите, где может возникнуть проблема

```
func Divide(a, b float64) (float64, error) {  
    if b == 0 {  
        return 0, errors.New("деление на ноль")  
    }  
    return a / b, nil  
}
```

Функции возвращают ошибку как последнее значение. Проверяйте её сразу после вызова

Создаем свои ошибки

Свои ошибки помогают точнее диагностировать проблему

```
var ErrFileNotFound = errors.New("файл не найден")
func OpenFile(name string) error {
    if _, err := os.Stat(name); err != nil {
        return fmt.Errorf("ошибка открытия: %w", ErrFileNotFound)
    }
    return nil
}
```

Пользовательские ошибки — как именованные сигналы тревоги. Если файл не найден, возвращаем `ErrFileNotFound`. А `fmt.Errorf` с `%w` добавляет контекст, например: «ошибка открытия: файл не найден». Это как написать на коробке «Осторожно, стекло!» вместо «Что-то разбилось»

Поиск ошибок в цепочке

errors.Is проверяет, есть ли конкретная ошибка в цепочке

```
err := OpenFile("config.yaml")
if errors.Is(err, ErrFileNotFound) {
    fmt.Println("Создаю новый файл...")
}
```

- Используйте errors.Is, чтобы проверить, является ли проверяемая ошибка исходной — он «распаковывает» ошибки, как будто разворачивает матрёшку, пока не найдёт нужную

Паника и восстановление

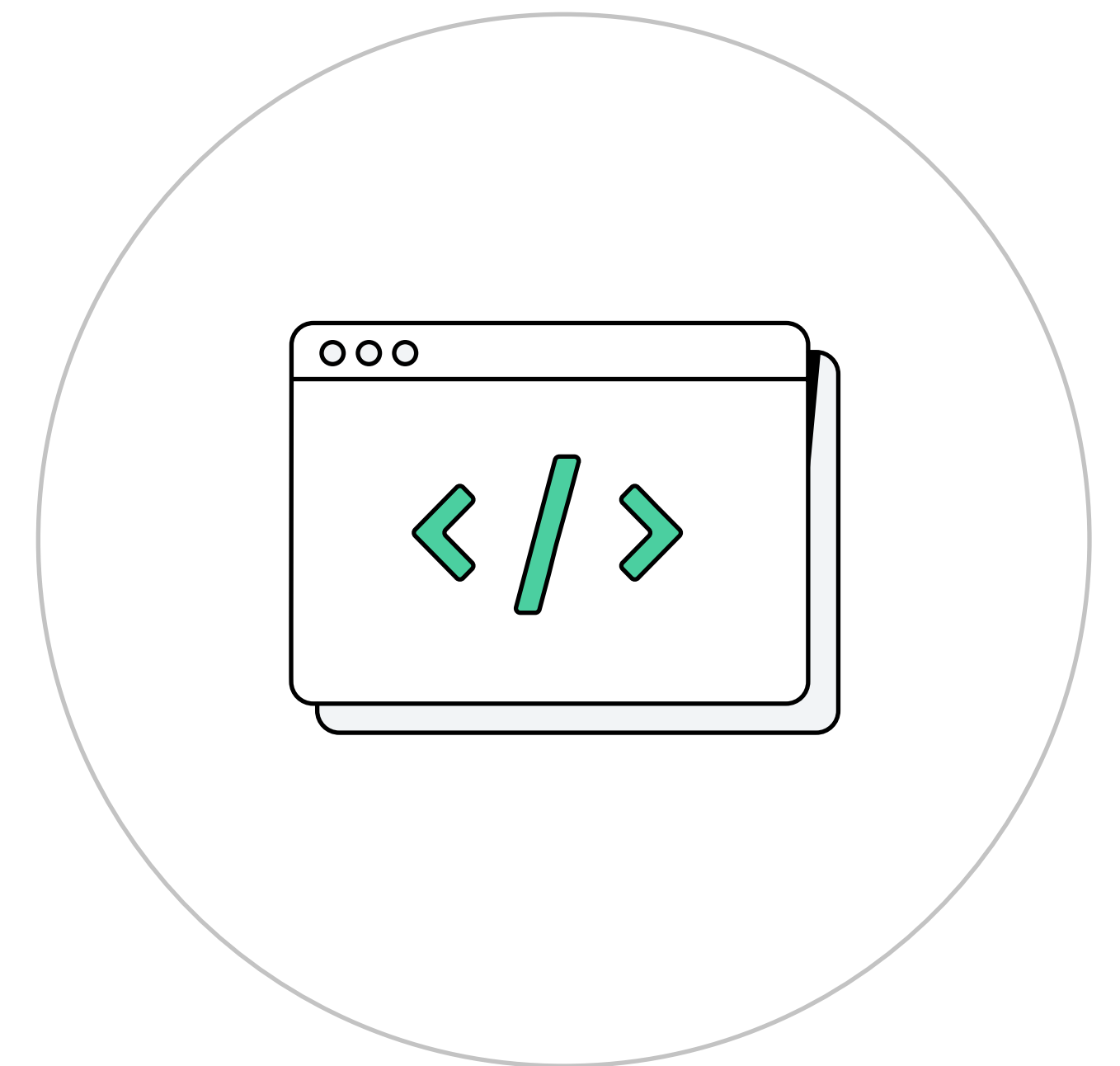
- panic останавливает программу
- recover перехватывает панику

```
func main() {  
    defer func() {  
        if r := recover(); r != nil {  
            fmt.Println("Программа восстановлена:", r)  
        }  
    }()  
    panic("все сломалось!")  
}
```

Паника и восстановление

Паника — это как крик «Пожар!» в здании. Все сразу останавливаются. Но если у вас есть ``recover`` в ``defer``, это как пожарная система, которая тушит огонь и позволяет продолжить работу

- Используйте это только для действительно критических ошибок, например, если база данных не подключилась при старте



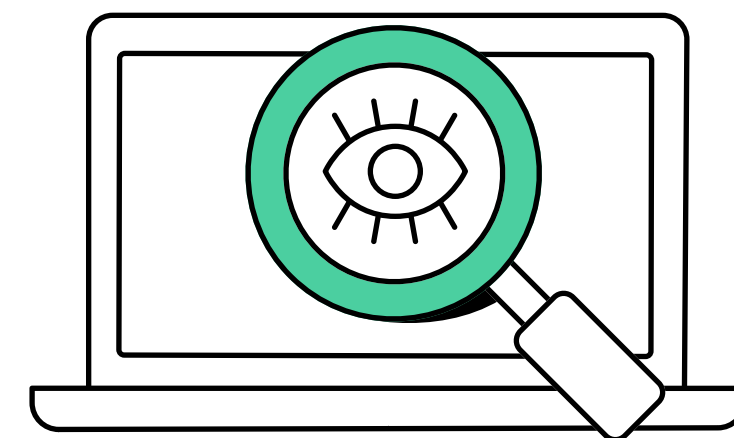
Лучшие практики

- Всегда добавляйте контекст к ошибкам
- Документируйте ошибки в описании функций
- Не игнорируйте ошибки через `\_`



Демонстрация работы

Обработка ошибок в языке Go



Итоги видео

- Узнали, как обрабатывать ошибки в языке Go
- Рассмотрели различных способы обработки ошибок и их применения в коде



Обработка ошибок в Go

Игорь Шамаев
Руководитель направления разработки

